

DATA STRUCTURES – DAY 4

PRACTICE QUESTIONS

75 Questions – Queues, Recursion & Backtracking, Hashing,
Trees, BST, Heaps, AVL & Red-Black Trees

Attempt without notes. Draw trees/heaps and dry-run rotations. Always state Time & Space complexity.

1–4. Queues

- Q1. Define Queue ADT. List the basic operations and their ideal time complexity.
- Q2. Why does a simple linear array queue waste space? How does a circular queue solve it?
- Q3. Write enqueue and dequeue for a circular queue (with modulo). How do you detect full vs empty?
- Q4. What is a Deque? Name two Java classes that implement it.
- Q5. How does PriorityQueue in Java order elements by default? How do you make a max-heap?
- Q6. Implement a queue using a linked list (keep front and rear). Why is enqueue $O(1)$?
- Q7. Implement a queue using two stacks. Analyse amortized complexity.
- Q8. Generate the first N binary numbers using a queue.

5–6. Recursion & Backtracking

- Q9. What is a base case? What happens without one?
- Q10. Write recursive Fibonacci. Why is it $O(2^n)$? How can memoization help?
- Q11. Explain Tower of Hanoi recurrence and number of moves for n disks.
- Q12. What is backtracking? How is it different from plain recursion?
- Q13. Write a backtracking solution to generate all subsets of an array.
- Q14. Write a backtracking solution to print all permutations of a string.
- Q15. Sketch the approach for N-Queens (place/remove + isSafe check).

7–9. Hashing

- Q16. What is a hash function? List three desirable properties.
- Q17. Explain chaining vs open addressing. Pros and cons of each.
- Q18. What is linear probing? What is primary clustering?
- Q19. What is the load factor? Why does HashMap resize when it exceeds a threshold?
- Q20. Solve Two-Sum in $O(n)$ time using a HashMap.
- Q21. Check whether an array contains duplicates using a HashSet. Complexity?
- Q22. Count character frequency in a string using HashMap.
- Q23. What is the average and worst-case complexity of search in a hash table?

10–12. Trees & Traversals

- Q24. Define: root, leaf, height, depth, degree, binary tree.
- Q25. Write recursive Preorder, Inorder and Postorder traversals.
- Q26. What order does Inorder give for a BST?
- Q27. Write level-order (BFS) traversal using a queue.
- Q28. Write a function to compute the height of a binary tree.
- Q29. Count the number of leaf nodes in a binary tree.
- Q30. Write iterative inorder traversal using a stack.
- Q31. Given preorder and inorder, can you reconstruct the binary tree? Explain briefly.

13–14. Binary Search Trees

- Q32. State the BST property. What is average and worst-case search complexity?
- Q33. Write recursive insert into a BST.
- Q34. Write recursive search in a BST.
- Q35. Explain the three cases of BST deletion. What is inorder successor?
- Q36. Validate whether a given binary tree is a BST (min/max range method).
- Q37. Find the kth smallest element in a BST.
- Q38. Find the Lowest Common Ancestor (LCA) of two nodes in a BST.
- Q39. Convert a sorted array into a height-balanced BST.

15–17. Heaps

- Q40. Define min-heap and max-heap. Give the index formulas for parent/left/right.
- Q41. What is the time complexity of insert and extract-min in a binary heap?

- Q42. Explain the heapify (sift-down) operation.
- Q43. How do you build a heap from an array in $O(n)$ time?
- Q44. Write / describe the Heap Sort algorithm. Is it stable? In-place?
- Q45. Find the K largest elements of an array using a min-heap of size K.
- Q46. How can two heaps be used to maintain the running median?
- Q47. Name three graph/algorithm applications that use priority queues / heaps.

18–19. Balanced Trees (AVL & Red-Black)

- Q48. What is the balance factor of an AVL node? What values are allowed?
- Q49. Describe the four AVL rotation cases: LL, RR, LR, RL.
- Q50. Why does AVL guarantee $O(\log n)$ for search/insert/delete?
- Q51. List the five Red-Black tree properties.
- Q52. How does a Red-Black tree differ from an AVL tree in terms of balance strictness?
- Q53. Which Java collection is implemented as a Red-Black tree?
- Q54. What problem occurs when you insert sorted data into an unbalanced BST?

Bonus / Mixed

- Q55. Implement a circular queue and demonstrate wrap-around with a dry-run.
- Q56. Design a stack using a single queue (or vice-versa). Complexity of push/pop?
- Q57. Solve the “sliding window maximum” problem using a Deque.
- Q58. Given a binary tree, print its left view / right view (use level order or recursion).
- Q59. Check if two binary trees are identical / mirror images of each other.
- Q60. Serialize and deserialize a binary tree (level-order or preorder with markers).
- Q61. Find the diameter of a binary tree (longest path between any two nodes).
- Q62. Connect nodes at the same level (next pointers) in a binary tree.
- Q63. Top-K frequent elements using a HashMap + Heap.
- Q64. Group anagrams together using a HashMap with sorted-key or count-key.
- Q65. Subarray sum equals K – solve with prefix-sum + HashMap.
- Q66. Dry-run Heap Sort on the array [4, 10, 3, 5, 1].
- Q67. Insert 1..7 in order into an empty BST and draw the resulting tree. Why is it skewed?
- Q68. After inserting the same sequence into an AVL tree, what does the tree look like?
- Q69. Explain why HashMap average $O(1)$ can degrade to $O(n)$ and how Java mitigates it (treeify).
- Q70. Implement “min stack” that supports push, pop, top and getMin in $O(1)$.
- Q71. Find the next greater element for every array element using a stack.
- Q72. Merge K sorted linked lists using a PriorityQueue. Complexity?
- Q73. Convert a BST to a sorted doubly linked list (in-place).
- Q74. What is the maximum number of nodes in a binary tree of height h? Minimum?
- Q75. Compare time complexities: unsorted array search vs BST vs HashSet vs balanced BST.

After solving, verify with the Day-4 Concept Guide. Draw every tree/heap operation and state complexities clearly.

All the best for your placements!